ARDUINO BASICS:

The Virtual Ardunio Board is found at: https://www.tinkercad.com/circuits

The basic Arduino program structure is:

```
//////variable setup/////



//////program setup/////


void setup()
{

   // put your setup code here, to run once:

}


///////program main body/////////////////

void loop()
{
   // put your main code here, to run repeatedly:
}
```

Defining Variables

int variable_name = integer;

ex: int myPin = 5; (sets the integer variable called pin to a value of 5);

boolean variable_name = TRUE (or FALSE) (or HIGH, or LOW);

ex: boolean itIsOn = FALSE;

Setup Functions

pinMode (variable_name, OUTPUT (or INPUT); //sets up a board pin as an input or output.

ex: pinMode (led, OUTPUT); // initializes the pin at the variable led as an output

Main Loop Functions

digitalWrite (variable_name, HIGH (or LOW)); // turns a pin on or off

ex: digitalWrite (led, HIGH);

digitalRead (variable_name, HIGH (or LOW); *//checks the status of a button or pin*

delay (1000); // wait for a certain amount of time 1000 units = 1 second

analogWrite(pin, PWM);

// This command sends pulse width modulation (PWM) to the pins with a ~ symbol. This will simulate sending of partial power through a digital (binary) channel. PWM ranges from 0 – 255. 255 is 100% power, 127 is 50% power, etc.

analogRead (pin);

//reads value coming into an analog pin. Generally, this data comes from a sensor.

map (value, input range low, input range high, output range low, output range high);

// This command takes a value from one range and transposes it into the same relative position in a second range. For example, if you input: map(50,0,100,200,600); the output will be 400. This is because 50 is exactly between 0 and 100 and 400 is exactly between 200 and 600.

CONDITIONALS

IF STATEMENT

```
if (someCondition) {
  // do stuff if the condition is true
}

else {
  // do stuff if the condition is false
}
```

// NOTE: the else in this case is optional. It can also lead to multiple nested if statements like this:

```
if (someCondition) {
  // do stuff if the condition is true
}

else if (anotherCondition) {
  // do stuff only if the first condition is false
  // and the second condition is true
}

else if (aThirdCondition) {
  // do stuff only if the first two conditions are false
  // and the third condition is true
}

else {
        // do stuff if all other conditions are false
}
```

****example:**

```
if (digitalRead (buttonPressed == HIGH)  //when using "Is equal to, use == instead of =

{

        digitalWrite (LED1, HIGH);

        //in this case, if the button is pressed, LED1 will light up

}
```

FOR LOOPS:

For loops are a way to do a loop a specific number of times. This loop can be tied to a variable which changes.

```
void loop()
{

   for (int i = 1; i <=5; i = i+1) //this loops 5 times.
   {
      //the commands happening in your loop go here.
   }
}
```

As you can see, there are 3 parameters to the for loop.

- The first defines the variable (int i = 1).
- The second tells the program how many times to loop (i <= 5 (in this case, while i is less than or equal to 5).
- The last tells the program how to increment the variable at the end of the loop. i=i+1 adds 1 to the value at the end of each pass through the loop.

Writing Your Own Functions!!

void functionName () *///this goes AFTER the main body!*

{

whatTheFunctionDoes ();

anotherThingThatTheFunctionDoes ();

}

///obviously, what the function does is some sort of command list that you want to be able to call in the main body more than once. To call it in the void loop (), you simply write ***functionName ();***

Interrupts

// Interrupts stop the main void loop when a trigger (such as a button press) occurs.

In void setup:

attachInterrupt (pin_#, function_to_interrupt_with, MODE);

Pin_# *can be 0 (for pin #2) or 1 (for pin #3). Those are the only pins that interrupt on the Arduino UNO.*

function_to_interrupt_with: *This is a user defined function. It can be called anything and can have no code (restarting the void loop, essentially) or an action that you want the program to take. The function should not have any delays in it.*

MODE: There are several possibilities depending on what you want to check for:

- **LOW** *to trigger the interrupt whenever the pin is low (button is not pressed).*
- **CHANGE** *to trigger the interrupt whenever the pin changes value (button is pressed OR released).*
- ***RISING*** *to trigger when the pin goes from low to high (Button is pressed).*
- **FALLING** *for when the pin goes from high to low. (Button is released (having been pressed)).*

Serial Monitor

//One good way to debug is to use the serial moniter. It basically allows your Arduino to write the state of different pins and variables to the screen along with text (also called “strings”).\

void setup ()

{

Serial.begin (9600); //sets up the serial monitor

}

void loop ()

{

Serial.print (variable_name);

// Prints the VALUE of a variable to the Serial monitor. It will not end the line, so each time the Serial.print is called, it will place the numbers one after another.

Serial.println (variable_name);

//Same as Serial.print, but puts a line break (like hitting enter) at the end of the line.

}

/// NOTE: *In both instances you can put text using* ***Serial.print (“HELLO WORLD”);*** *instead of a variable name. That will write the text HELLO WORLD in the serial monitor.*

Example: *You want to find out how many times someone has clicked a button and you have a variable called buttonClicks that increases every time it is clicked (pretend the button has been pressed 3 times).*

If you use the code:

```
void loop ()
{
        Serial.print ("You have clicked the button ");
        Serial.print (buttonClicks);
        Serial.println (" times!");
}
```

The serial monitor will print out:

You have clicked the button 3 times!

It will then drop to the next line in the Serial Monitor (because of the Serial.println).

SERVO CONTROL

//hardware: Wire the servo with 5V, GND and a Digital control pin.

//in the variable section:

```
#include <Servo.h>
Servo myServo;      //"myServo" is the servo name and can be anything
```

//in void setup ()

```
  myServo.attach (pinNumber);
```

//in void loop()

```
myServo.write (angle);  //Moves the servo to a specific angle position, generally this will be 0-180
```